

```
In [1]: print("Python is working!")
```

Python is working!

```
In [2]: import pandas as pd

df = pd.read_csv(r"C:\Users\HP\Desktop\ecommerce_sales.csv")

df.head()
```

 FileNotFoundError

Traceback (most recent call last)

Cell In[2], line 3

```
1 import pandas as pd
2
----> 3 df = pd.read_csv(r"C:\Users\HP\Desktop\ecommerce_sales.csv")
4
5 df.head()
```

File ~\anaconda3\Lib\site-packages\pandas\io\parsers\readers.py:873, in read_csv(filepath_or_buffer, sep, delimiter, header, names, index_col, usecols, dtype, engine, converters, true_values, false_values, skipinitialspace, skiprows, skipfooter, nrows, na_values, keep_default_na, na_filter, skip_blank_lines, parse_dates, date_format, dayfirst, cache_dates, iterator, chunksize, compression, thousands, decimal, lineterminator, quotechar, quoting, doublequote, escapechar, comment, encoding, encoding_errors, dialect, on_bad_lines, low_memory, memory_map, float_precision, storage_options, dtype_backend)

```
861 kws_defaults = _refine_defaults_read(
862     dialect,
863     delimiter,
(...) 869     dtype_backend=dtype_backend,
870 )
871 kws.update(kws_defaults)
--> 873 return _read(filepath_or_buffer, kws)
```

File ~\anaconda3\Lib\site-packages\pandas\io\parsers\readers.py:300, in _read(filepath_or_buffer, kws)

```
297 _validate_names(kws.get("names", None))
299 # Create the parser.
--> 300 parser = TextFileReader(filepath_or_buffer, **kws)
302 if chunksize or iterator:
303     return parser
```

File ~\anaconda3\Lib\site-packages\pandas\io\parsers\readers.py:1645, in TextFileReader.__init__(self, f, engine, **kws)

```
1642 self.options["has_index_names"] = kws["has_index_names"]
1644 self.handles: IOHandles | None = None
-> 1645 self._engine = self._make_engine(f, self.engine)
```

File ~\anaconda3\Lib\site-packages\pandas\io\parsers\readers.py:1904, in TextFileReader._make_engine(self, f, engine)

```
1902 if "b" not in mode:
1903     mode += "b"
-> 1904 self.handles = get_handle(
1905     f,
1906     mode,
1907     encoding=self.options.get("encoding", None),
1908     compression=self.options.get("compression", None),
1909     memory_map=self.options.get("memory_map", False),
1910     is_text=is_text,
1911     errors=self.options.get("encoding_errors", "strict"),
1912     storage_options=self.options.get("storage_options", None),
1913 )
1914 assert self.handles is not None
1915 f = self.handles.handle
```

```
File ~\anaconda3\Lib\site-packages\pandas\io\common.py:930, in get_handle(path_or_buf, mode, encoding, compression, memory_map, is_text, errors, storage_options)
    925 elif isinstance(handle, str):
    926     # Check whether the filename is to be opened in binary mode.
    927     # Binary mode does not support 'encoding' and 'newline'.
    928     if ioargs.encoding and "b" not in ioargs.mode:
    929         # Encoding
--> 930         handle = open(
    931             handle,
    932             ioargs.mode,
    933             encoding=ioargs.encoding,
    934             errors=errors,
    935             newline="",
    936         )
    937     else:
    938         # Binary mode
    939         handle = open(handle, ioargs.mode)

FileNotFoundError: [Errno 2] No such file or directory: 'C:\\Users\\HP\\Desktop\\ecommerce_sales.csv'
```

```
In [3]: import pandas as pd

df = pd.read_csv(r"C:\Users\HP\Desktop\ecommerce_sales.csv")

df.head()
```

 FileNotFoundError

Traceback (most recent call last)

Cell In[3], line 3

```
1 import pandas as pd
2
----> 3 df = pd.read_csv(r"C:\Users\HP\Desktop\ecommerce_sales.csv")
4
5 df.head()
```

File ~\anaconda3\Lib\site-packages\pandas\io\parsers\readers.py:873, in read_csv(filepath_or_buffer, sep, delimiter, header, names, index_col, usecols, dtype, engine, converters, true_values, false_values, skipinitialspace, skiprows, skipfooter, nrows, na_values, keep_default_na, na_filter, skip_blank_lines, parse_dates, date_format, dayfirst, cache_dates, iterator, chunksize, compression, thousands, decimal, lineterminator, quotechar, quoting, doublequote, escapechar, comment, encoding, encoding_errors, dialect, on_bad_lines, low_memory, memory_map, float_precision, storage_options, dtype_backend)

```
861 kwds_defaults = _refine_defaults_read(
862     dialect,
863     delimiter,
(...) 869     dtype_backend=dtype_backend,
870 )
871 kwds.update(kwds_defaults)
--> 873 return _read(filepath_or_buffer, kwds)
```

File ~\anaconda3\Lib\site-packages\pandas\io\parsers\readers.py:300, in _read(filepath_or_buffer, kwds)

```
297 _validate_names(kwds.get("names", None))
299 # Create the parser.
--> 300 parser = TextFileReader(filepath_or_buffer, **kwds)
302 if chunksize or iterator:
303     return parser
```

File ~\anaconda3\Lib\site-packages\pandas\io\parsers\readers.py:1645, in TextFileReader.__init__(self, f, engine, **kwds)

```
1642 self.options["has_index_names"] = kwds["has_index_names"]
1644 self.handles: IOHandles | None = None
-> 1645 self._engine = self._make_engine(f, self.engine)
```

File ~\anaconda3\Lib\site-packages\pandas\io\parsers\readers.py:1904, in TextFileReader._make_engine(self, f, engine)

```
1902 if "b" not in mode:
1903     mode += "b"
-> 1904 self.handles = get_handle(
1905     f,
1906     mode,
1907     encoding=self.options.get("encoding", None),
1908     compression=self.options.get("compression", None),
1909     memory_map=self.options.get("memory_map", False),
1910     is_text=is_text,
1911     errors=self.options.get("encoding_errors", "strict"),
1912     storage_options=self.options.get("storage_options", None),
1913 )
1914 assert self.handles is not None
1915 f = self.handles.handle
```

```

File ~\anaconda3\Lib\site-packages\pandas\io\common.py:930, in get_handle(path_or_buf, mode, encoding, compression, memory_map, is_text, errors, storage_options)
    925 elif isinstance(handle, str):
    926     # Check whether the filename is to be opened in binary mode.
    927     # Binary mode does not support 'encoding' and 'newline'.
    928     if ioargs.encoding and "b" not in ioargs.mode:
    929         # Encoding
--> 930         handle = open(
    931             handle,
    932             ioargs.mode,
    933             encoding=ioargs.encoding,
    934             errors=errors,
    935             newline="",
    936         )
    937     else:
    938         # Binary mode
    939         handle = open(handle, ioargs.mode)

FileNotFoundError: [Errno 2] No such file or directory: 'C:\\Users\\HP\\Desktop\\ecommerce_sales.csv'

```

```

In [4]: import os

os.listdir(r"C:\Users\HP\Desktop")

```

```

-----
FileNotFoundError                                Traceback (most recent call last)
Cell In[4], line 3
      1 import os
      2
----> 3 os.listdir(r"C:\Users\HP\Desktop")

FileNotFoundError: [WinError 3] The system cannot find the path specified: 'C:\\Users\\HP\\Desktop'

```

```

In [5]: df = pd.read_csv(r"C:\Users\HP\OneDrive\Desktop\ecommerce_sales.csv")

df.head()

```

Out[5]:

	Order ID	Order Date	Months	Customer Name	Region	Category	Product	Quantity
0	ORD0001	2026-07-08 00:00:00	July	Nimali Silva	Sabaragamuwa	Electronics	Laptop	6
1	ORD0002	2026-06-04 00:00:00	June	Kasun Perera	Sabaragamuwa	Stationery	Notebook	1
2	ORD0003	2026-01-02 00:00:00	January	Amaya Fernando	North Central	Stationery	Pen Set	4
3	ORD0004	2026-01-24 00:00:00	January	Piumi Silva	Central	Accessories	Mouse	2
4	ORD0005	2026-04-07 00:00:00	April	Nimali Silva	Sabaragamuwa	Accessories	USB Cable	1

In [6]: df.info()

```
<class 'pandas.DataFrame'>
RangeIndex: 150 entries, 0 to 149
Data columns (total 13 columns):
#   Column                Non-Null Count  Dtype
---  -
0   Order ID              150 non-null   str
1   Order Date            150 non-null   str
2   Months                150 non-null   str
3   Customer Name         150 non-null   str
4   Region                150 non-null   str
5   Category              150 non-null   str
6   Product               150 non-null   str
7   Quantity              150 non-null   int64
8   Unit Price            150 non-null   float64
9   Sales                 150 non-null   float64
10  Sales Check           150 non-null   float64
11  Profit                150 non-null   float64
12  Payment Method        150 non-null   str
dtypes: float64(4), int64(1), str(8)
memory usage: 26.7 KB
```

In [7]: df.isnull().sum()

```
Out[7]: Order ID      0
        Order Date   0
        Months       0
        Customer Name 0
        Region       0
        Category     0
        Product      0
        Quantity     0
        Unit Price    0
        Sales        0
        Sales Check   0
        Profit       0
        Payment Method 0
        dtype: int64
```

```
In [8]: df.duplicated().sum()
```

```
Out[8]: np.int64(0)
```

```
In [9]: df["Order Date"] = pd.to_datetime(df["Order Date"])
```

```
In [10]: df["Order Date"].dtype
```

```
Out[10]: dtype('<M8[us]')
```

```
In [11]: df["Sales_Check_Python"] = df["Quantity"] * df["Unit Price"]

(df["Sales_Check_Python"] == df["Sales"]).all()
```

```
Out[11]: np.False_
```

```
In [12]: df["Difference"] = df["Sales_Check_Python"] - df["Sales"]

df[df["Difference"].abs() > 0.01][
    ["Order ID", "Quantity", "Unit Price", "Sales", "Sales_Check_Python", "Difference"]
]
```

```
Out[12]:
```

Order ID	Quantity	Unit Price	Sales	Sales_Check_Python	Difference
----------	----------	------------	-------	--------------------	------------

```
In [13]: import numpy as np

np.allclose(df["Sales_Check_Python"], df["Sales"], atol=0.01)
```

```
Out[13]: True
```

```
In [14]: df.describe()
```

Out[14]:

	Order Date	Quantity	Unit Price	Sales	Sales Check	Profit	Sa
count	150	150.000000	150.000000	1.500000e+02	1.500000e+02	150.000000	
mean	2026-04-26 08:19:12	3.393333	31587.140600	1.165889e+05	1.165889e+05	19966.428200	
min	2026-01-02 00:00:00	1.000000	637.580000	8.897600e+02	8.897600e+02	139.500000	
25%	2026-03-05 06:00:00	2.000000	1725.567500	4.558288e+03	4.558288e+03	906.092500	
50%	2026-04-20 12:00:00	3.000000	11749.280000	2.918206e+04	2.918206e+04	5371.610000	
75%	2026-06-21 18:00:00	5.000000	48923.707500	1.552814e+05	1.552814e+05	22316.742500	
max	2026-08-19 00:00:00	6.000000	193852.380000	1.163114e+06	1.163114e+06	267720.480000	
std	NaN	1.646215	44780.470497	1.989738e+05	1.989738e+05	37970.931985	

In [15]: `df.drop(columns=["Sales_Check_Python", "Difference"], inplace=True)`

In [16]: `df.info()`

```
<class 'pandas.DataFrame'>
RangeIndex: 150 entries, 0 to 149
Data columns (total 13 columns):
#   Column                Non-Null Count  Dtype
---  -
0   Order ID              150 non-null   str
1   Order Date            150 non-null   datetime64[us]
2   Months                150 non-null   str
3   Customer Name         150 non-null   str
4   Region                150 non-null   str
5   Category              150 non-null   str
6   Product               150 non-null   str
7   Quantity              150 non-null   int64
8   Unit Price            150 non-null   float64
9   Sales                 150 non-null   float64
10  Sales Check           150 non-null   float64
11  Profit                150 non-null   float64
12  Payment Method        150 non-null   str
dtypes: datetime64[us](1), float64(4), int64(1), str(7)
memory usage: 23.9 KB
```

```
In [17]: total_sales = df["Sales"].sum()

print("Total Sales:", total_sales)
```

Total Sales: 17488334.75

```
In [18]: total_profit = df["Profit"].sum()

print("Total Profit:", total_profit)
```

Total Profit: 2994964.2299999995

```
In [19]: total_orders = df["Order ID"].count()

print("Total Orders:", total_orders)
```

Total Orders: 150

```
In [20]: average_order_value = df["Sales"].mean()

print("Average Order Value:", average_order_value)
```

Average Order Value: 116588.89833333333

```
In [21]: category_sales = (
    df.groupby("Category")["Sales"]
      .sum()
      .sort_values(ascending=False)
    )

print(category_sales)
```

Category
Electronics 11967114.64
Home 3837723.47
Accessories 1506274.04
Stationery 177222.60
Name: Sales, dtype: float64

```
In [22]: region_sales = (
    df.groupby("Region")["Sales"]
      .sum()
      .sort_values(ascending=False)
    )

print(region_sales)
```

Region	
Central	3120959.78
Western	2747591.61
Northern	2647136.28
Sabaragamuwa	2572876.30
North Central	1663481.27
Uva	1492727.35
Eastern	1193245.43
Southern	1121452.74
North Western	928863.99

Name: Sales, dtype: float64

```
In [23]: payment_sales = (
    df.groupby("Payment Method")["Sales"]
        .sum()
        .sort_values(ascending=False)
    )

    print(payment_sales)
```

Payment Method	
Bank Transfer	5015659.64
Card	4995739.30
Cash	4720374.02
Online	2756561.79

Name: Sales, dtype: float64

```
In [24]: top_products = (
    df.groupby("Product")["Sales"]
        .sum()
        .sort_values(ascending=False)
        .head(10)
    )

    print(top_products)
```

Product	
Laptop	6002971.20
Smartphone	4117179.97
Desk	1967290.31
Monitor	1603154.65
Office Chair	1262873.30
Webcam	862368.29
Bookshelf	428643.77
Keyboard	416864.33
Headphones	243808.82
Mouse	210007.57

Name: Sales, dtype: float64

```
In [25]: monthly_sales = (
    df.groupby(df["Order Date"].dt.to_period("M"))["Sales"]
        .sum()
    )

    print(monthly_sales)
```

```

Order Date
2026-01    1708483.50
2026-02    2460923.15
2026-03    2649937.57
2026-04    1905233.71
2026-05    2467413.82
2026-06    2091124.75
2026-07    3656681.73
2026-08     548536.52
Freq: M, Name: Sales, dtype: float64

```

```

In [26]: profit_by_category = (
    df.groupby("Category")["Profit"]
        .sum()
        .sort_values(ascending=False)
    )

print(profit_by_category)

```

```

Category
Electronics    2063589.60
Home            642235.67
Accessories    255863.54
Stationery      33275.42
Name: Profit, dtype: float64

```

```

In [27]: top_customers = (
    df.groupby("Customer Name")["Sales"]
        .sum()
        .sort_values(ascending=False)
        .head(10)
    )

print(top_customers)

```

```

Customer Name
Nimali Silva    3088020.78
Amaya Fernando  2669073.17
Tharushi Perera 2062506.01
Dilan Jayasinghe 1971146.04
Chamod Senanayake 1956929.78
Sahan Kumara    1420044.93
Piumi Silva     1410672.70
Nethmi Wijesinghe 1395873.86
Kasun Perera    930069.13
Ravindu Fernando 583998.35
Name: Sales, dtype: float64

```

```

In [28]: top_profit_products = (
    df.groupby("Product")["Profit"]
        .sum()
        .sort_values(ascending=False)
        .head(10)
    )

print(top_profit_products)

```

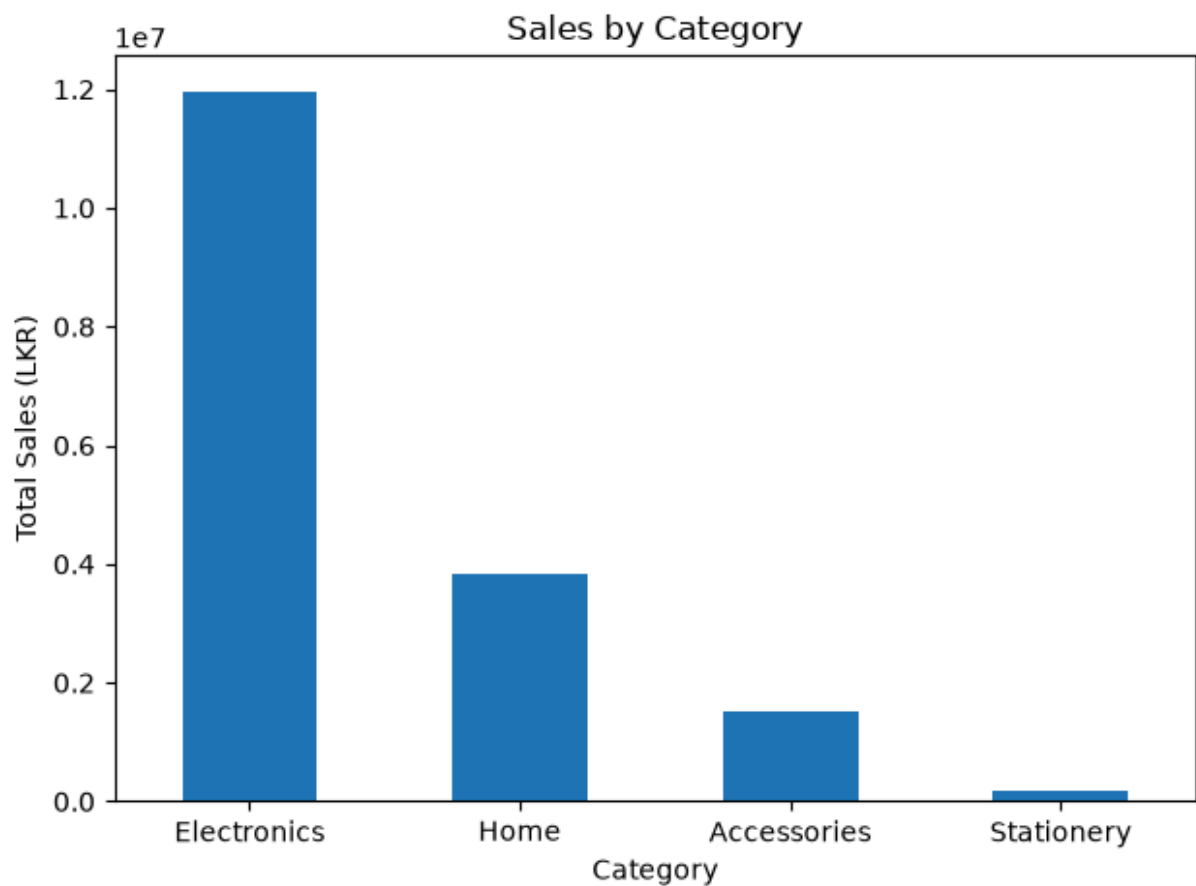
Product	
Laptop	1030003.51
Smartphone	693721.74
Monitor	300049.51
Desk	292396.91
Office Chair	265481.88
Webcam	152600.86
Keyboard	63850.22
Bookshelf	55297.08
Headphones	39814.84
Mouse	36085.29

Name: Profit, dtype: float64

```
In [29]: import matplotlib.pyplot as plt

category_sales.plot(kind="bar")

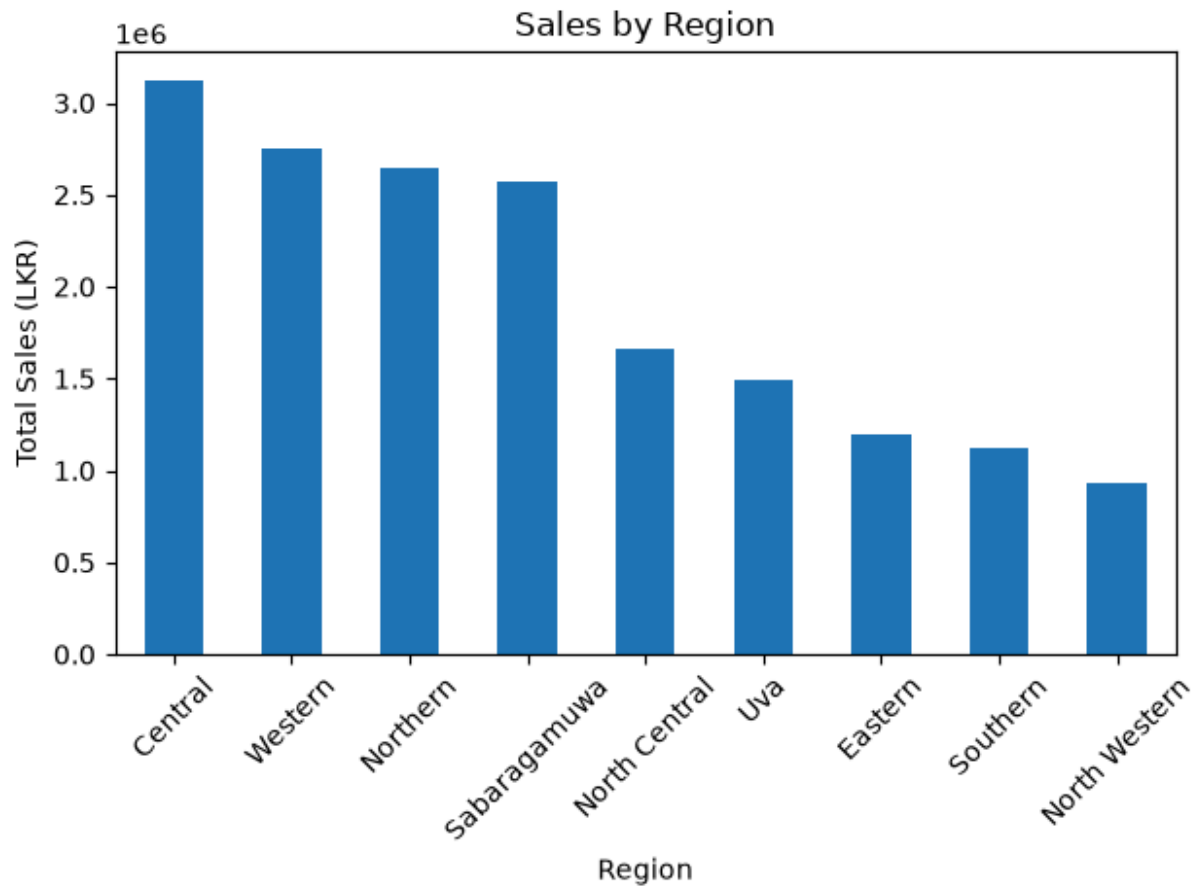
plt.title("Sales by Category")
plt.xlabel("Category")
plt.ylabel("Total Sales (LKR)")
plt.xticks(rotation=0)
plt.tight_layout()
plt.show()
```



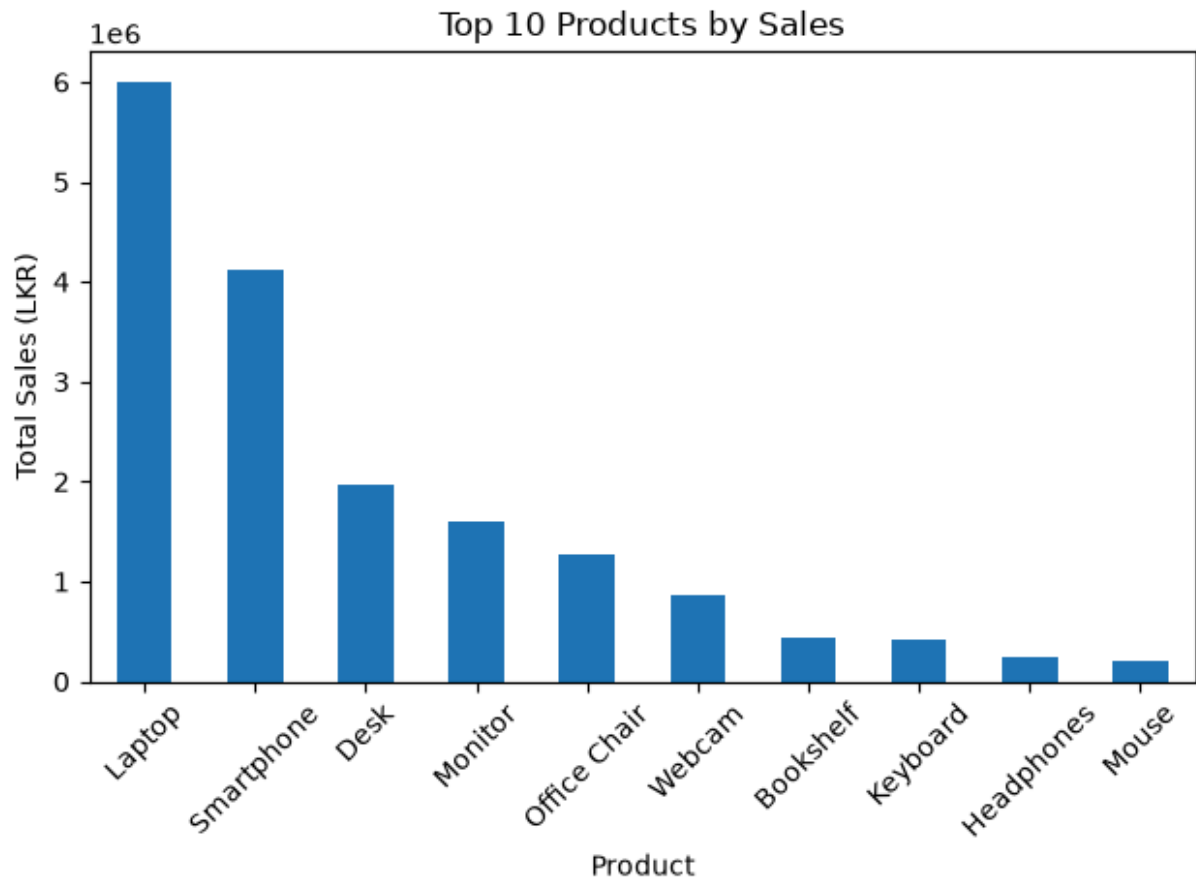
```
In [30]: region_sales.plot(kind="bar")

plt.title("Sales by Region")
plt.xlabel("Region")
```

```
plt.ylabel("Total Sales (LKR)")  
plt.xticks(rotation=45)  
plt.tight_layout()  
plt.show()
```

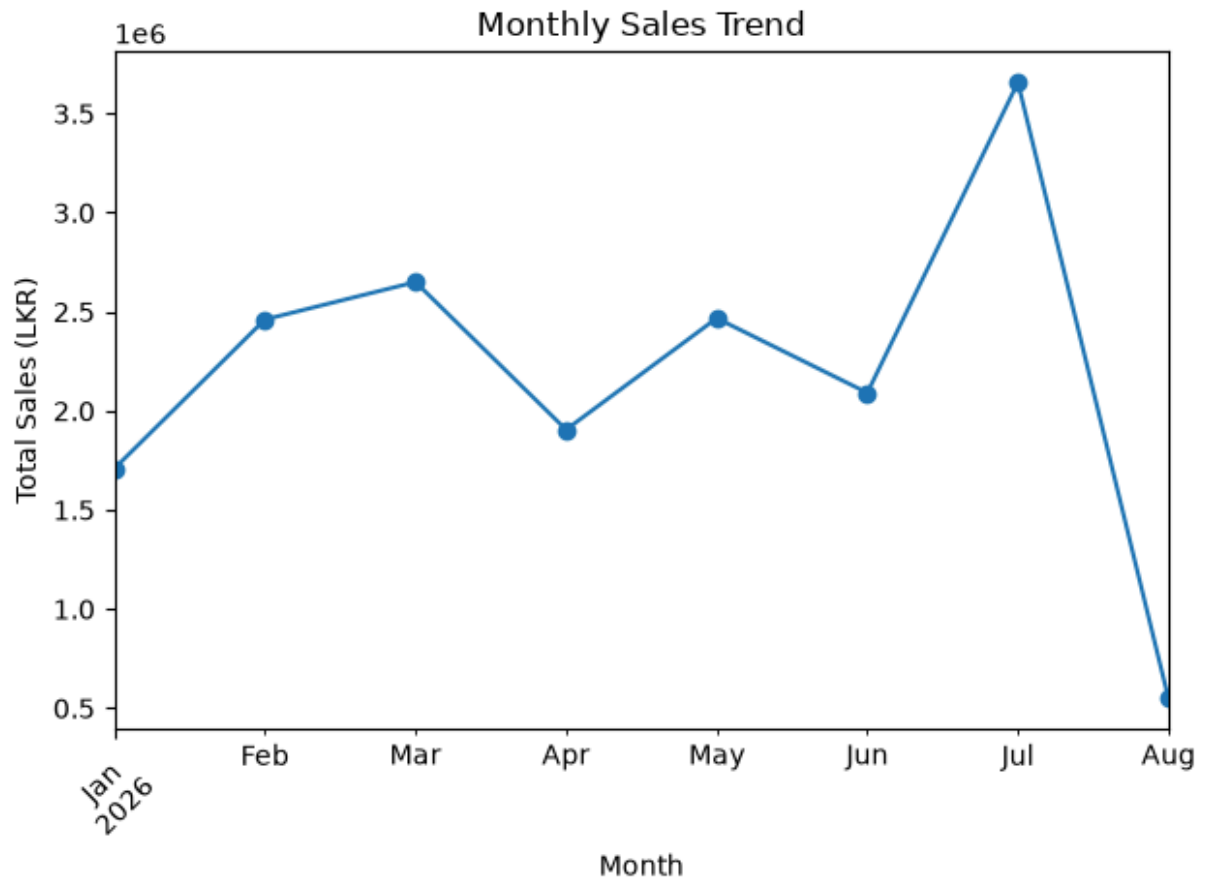


```
In [31]: top_products.plot(kind="bar")  
  
plt.title("Top 10 Products by Sales")  
plt.xlabel("Product")  
plt.ylabel("Total Sales (LKR)")  
plt.xticks(rotation=45)  
plt.tight_layout()  
plt.show()
```



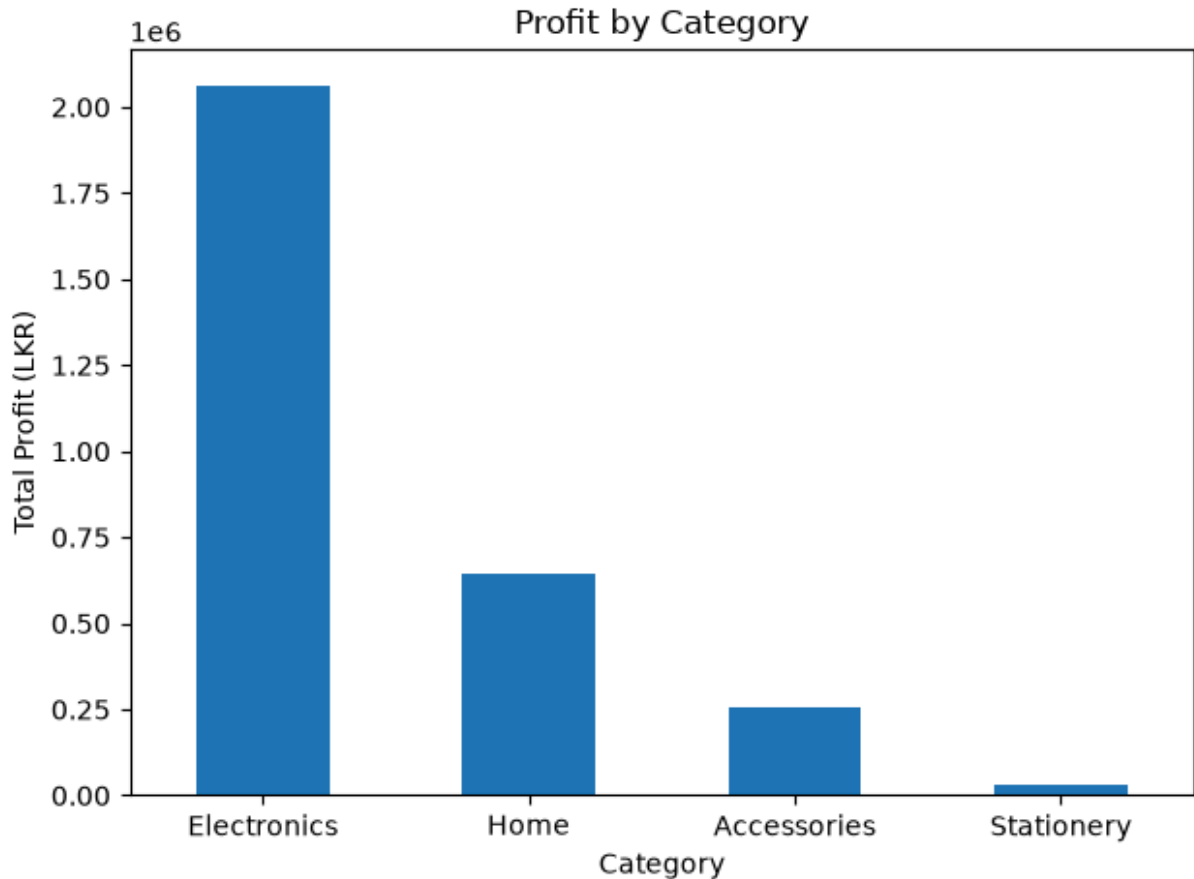
```
In [32]: monthly_sales.plot(kind="line", marker="o")
```

```
plt.title("Monthly Sales Trend")
plt.xlabel("Month")
plt.ylabel("Total Sales (LKR)")
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```



```
In [33]: profit_by_category.plot(kind="bar")
```

```
plt.title("Profit by Category")
plt.xlabel("Category")
plt.ylabel("Total Profit (LKR)")
plt.xticks(rotation=0)
plt.tight_layout()
plt.show()
```



```
In [34]: print("Business Insights")
print("-----")

print("1. Highest Sales Category:", category_sales.idxmax())
print("2. Highest Sales Region:", region_sales.idxmax())
print("3. Highest Sales Product:", top_products.index[0])
print("4. Highest Sales Payment Method:", payment_sales.idxmax())
print("5. Highest Profit Category:", profit_by_category.idxmax())
print("6. Total Sales:", round(total_sales, 2))
print("7. Total Profit:", round(total_profit, 2))
print("8. Total Orders:", total_orders)
print("9. Average Order Value:", round(average_order_value, 2))
```

Business Insights

1. Highest Sales Category: Electronics
2. Highest Sales Region: Central
3. Highest Sales Product: Laptop
4. Highest Sales Payment Method: Bank Transfer
5. Highest Profit Category: Electronics
6. Total Sales: 17488334.75
7. Total Profit: 2994964.23
8. Total Orders: 150
9. Average Order Value: 116588.9

```
In [ ]: pip install "nbconvert[webpdf]"
```

```
In [ ]: jupyter nbconvert --version
```

```
In [ ]:
```